

Cal Poly Pomona

ETE 4201 Fall 25

Professor Herder

December 4, 2025



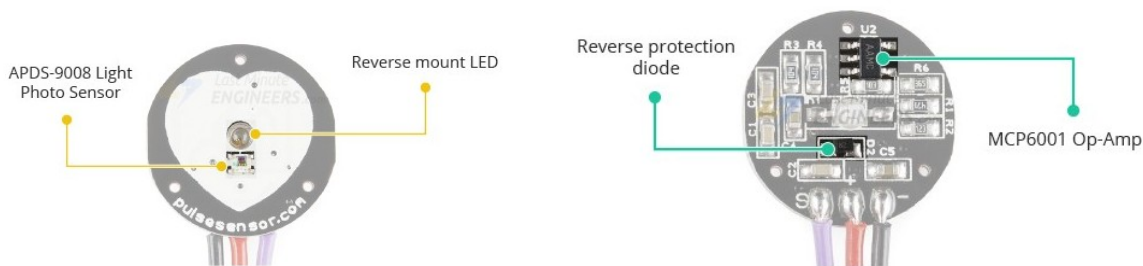
Sensor Project #2

Heart Rate Monitor

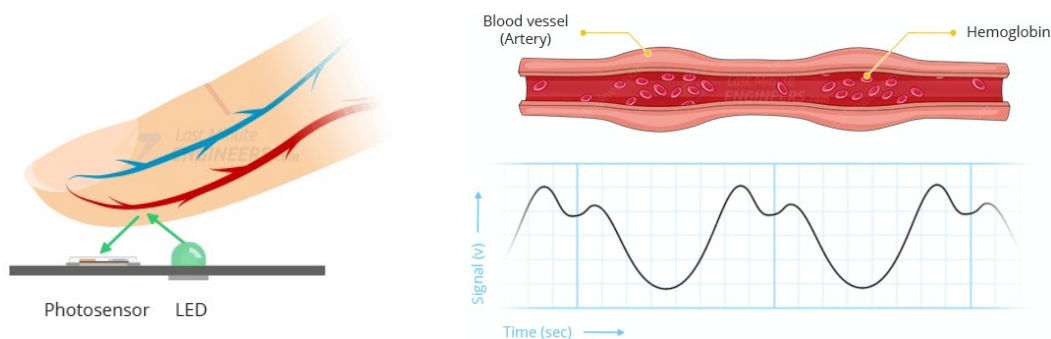
Prepared by: Mariana Rivera, Wesley Hwee, Sarah Boyd

Introduction: This project implements a basic heart rate monitoring system using an Arduino Uno and an LCD screen. The sensor uses photoplethysmography (PPG) to detect pulse by measuring small changes in light intensity caused by blood-volume variations in the fingertip. The sensor's signal, power, and ground pins are interfaced to an Arduino analog input (e.g., A0), 5 V, and GND, respectively. The Arduino samples and filters the analog waveform, detects heartbeat peaks, and computes beats per minute (BPM) from the time between beats. Real-time feedback is provided through the LCD display for BPM output and an LED indicator that blinks with each detected pulse.

Hardware Overview:

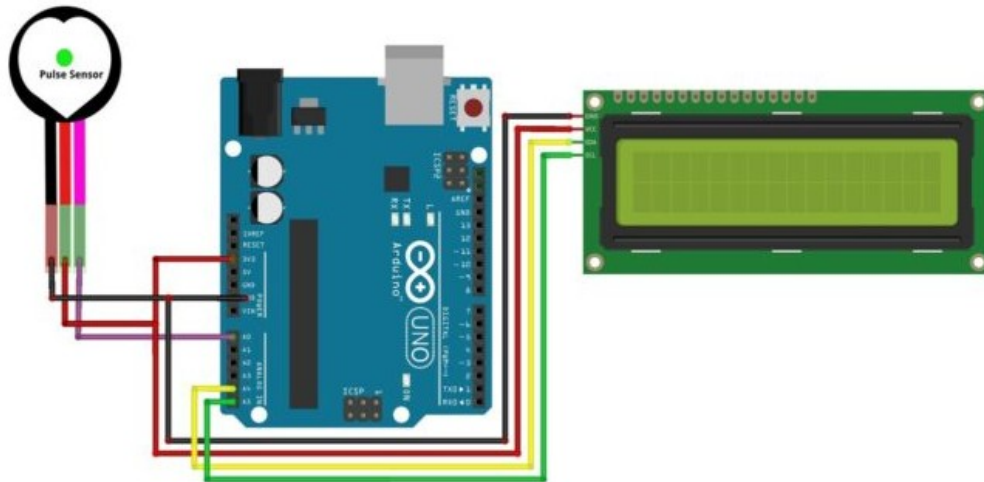


How does it work?

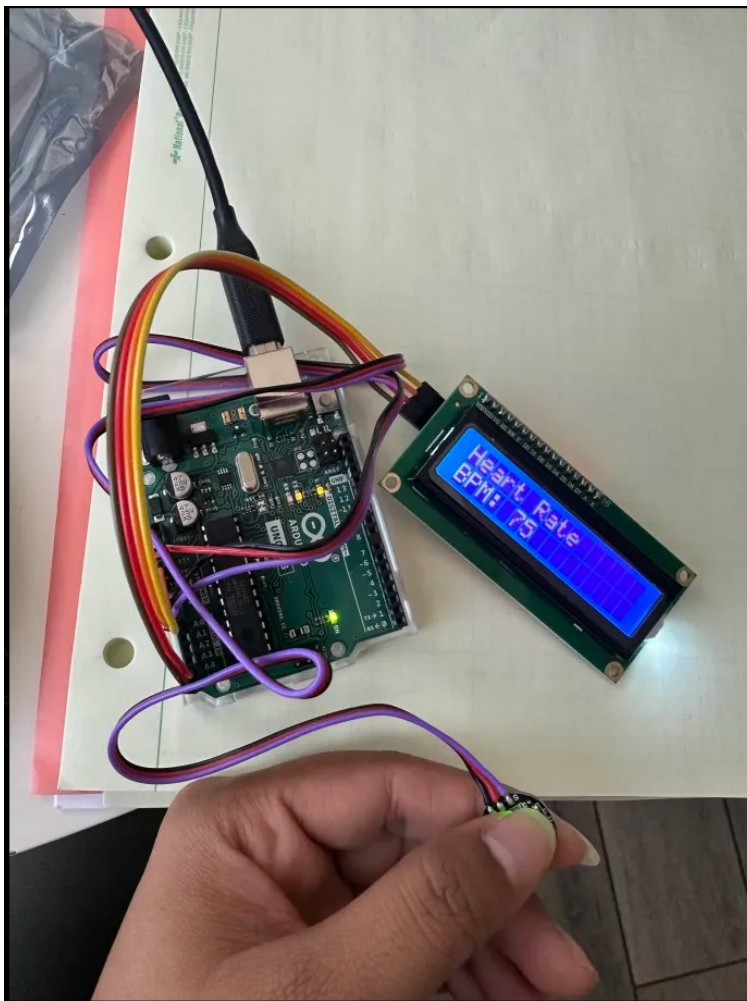


A pulse sensor measures heart rate using photoplethysmography (PPG), the same principle observed when light shines through a fingertip. The sensor's LED illuminates a thin tissue region (e.g., fingertip), and a photodetector measures the reflected light intensity. As the heart pumps blood, the local blood volume changes, altering how much green light is absorbed, primarily by oxygenated hemoglobin—so the reflected light decreases during a pulse peak and increases between beats. These periodic changes produce a waveform correlated with the heartbeat. Because the raw photodetector signal is small and susceptible to noise, it is typically conditioned using an RC filter for noise reduction and an operational amplifier to increase amplitude for reliable heartbeat detection and BPM calculation.

Hardware Wiring Diagram:



Built Circuit:



Video Demonstrating the Heart Rate Sensor working compared to my Smart Watch.



Code for Arduino Uno:

```
HeartRate_ICino
1  #define USE_ARDUINO_INTERRUPTS true
2  #include <PulseSensorPlayground.h>
3  #include <LiquidCrystal_I2C.h>
4  LiquidCrystal_I2C lcd(0x27, 16, 2); // set the LCD address to 0x27 for a 16 chars and 2 line display
5
6
7  // Constants
8  const int PULSE_SENSOR_PIN = 0; // Analog PIN where the PulseSensor is connected
9  const int LED_PIN = 13; // On-board LED PIN
10 const int THRESHOLD = 550; // Threshold for detecting a heartbeat
11
12 // Create PulseSensorPlayground object
13 PulseSensorPlayground pulseSensor;
14
15 void setup()
16 {
17     // Initialize Serial Monitor
18     Serial.begin(9600);
19     lcd.init();
20     lcd.backlight();
21
22     // Configure PulseSensor
23     pulseSensor.analogInput(PULSE_SENSOR_PIN);
24     pulseSensor.blinkOnPulse(LED_PIN);
25     pulseSensor.setThreshold(THRESHOLD);
26
27     // Check if PulseSensor is initialized
28     if (pulseSensor.begin())
29     {
30         Serial.println("PulseSensor object created successfully!");
```

```

HeartRate_IC.ino
27 // Check if PulseSensor is initialized
28 if (pulseSensor.begin())
29 {
30     Serial.println("PulseSensor object created successfully!");
31 }
32 }
33
34 void loop()
35 {
36     lcd.setCursor(0, 0);
37     lcd.print("Heart Rate");
38
39     // Get the current Beats Per Minute (BPM)
40     int currentBPM = pulseSensor.getBeatsPerMinute();
41
42     // Check if a heartbeat is detected
43     if (pulseSensor.sawStartOfBeat())
44     {
45         Serial.print("BPM : ");
46         Serial.println(currentBPM);
47
48         lcd.clear();
49         lcd.setCursor(0, 1);
50         lcd.print("BPM: ");
51         lcd.print(currentBPM);
52     }
53 }
54
55 // Add a small delay to reduce CPU usage
56 delay(20);

```

What did we learn from this?

By building and coding the pulse sensor system, we learned how a real-world bio-signal is captured, conditioned, and processed into a usable measurement. Wiring the sensor to the Arduino reinforced proper analog interfacing (power, ground, and reading the signal on an analog input), and it made it clear how sensitive the PPG signal is to motion, pressure on the fingertip, and ambient light. On the software side, we learned how to sample the signal consistently, apply basic filtering/smoothing, and detect heartbeat peaks reliably enough to calculate BPM from the time between beats. Integrating the LED and display output also helped connect the code to make it easier to see each detected beat produced with an updated BPM. This project showed us how sensor data becomes a real-time instrument readout and how these sensors work.

Conclusion: This lab demonstrated how a pulse sensor can estimate heart rate using photoplethysmography (PPG). By shining light into tissue and measuring the reflected intensity, the sensor detected periodic changes in absorption caused by pulsatile blood-volume variation with each heartbeat. The resulting waveform corresponded to the cardiac cycle, but the raw photodetector output required conditioning; RC filtering reduced noise and op-amp amplification

increased signal magnitude to support reliable peak detection. Overall, the results confirmed that optical sensing combined with basic analog signal conditioning provides a practical method for heartbeat detection and BPM measurement.

References:

<https://how2electronics.com/pulse-rate-bpm-monitor-arduino-pulse-sensor/>

<https://lastminuteengineers.com/pulse-sensor-arduino-tutorial/>

<https://techatronic.com/heart-beat-sensor-using-arduino-bpm-monitor/>